HS BO Hochschule Bochum	Hochschule Bochum Bochum University of Applied Sciences

Prüfungsform:	**Klausur**
Prüfungsfach:	**Objektorientierte Programmierung, Objektorientierte Programmiertechniken, Softwaretechnik und - Architektur 2 für Bachelor Informatik und Bachelor Wirtschaftsinformatik**
Semester:	**WiSe 2022 / 2023**
Datum:	**26.01.2023**
Prüfer:	**Prof. Dr. Ursula Oesing**
Bearbeitungsdauer:	**120 Minuten**
Hilfsmittel:	**Es sind keine eigenen Hilfsmittel zugelassen.**

Kandidat / Kandidatin:

Vorname:	……
Nachname:	……
Matrikelnummer:	……
Unterschrift:	……

Punkte: Es können 120 Punkte erreicht werden.

Hinweise:

Die 120 Punkte werden in Prozentanteile umgerechnet. Bitte schreiben Sie alle Antworten auf die Aufgabenblätter. Wenn der gegebene Platz unter der Fragestellung nicht ausreicht, können Sie auch die Rückseiten der Aufgabenblätter benutzen. Bitte kennzeichnen Sie jeweils eindeutig, zu welcher Frage ihre Antwort gehört! Unleserliche Bestandteile und Bestandteile auf selbst mitgebrachtem Papier werden nicht gewertet.

Mit der Annahme der Aufgabenstellung erklären Sie sich gesundheitlich in der Lage, die Klausur mitzuschreiben. Bei Täuschungsversuchen wird die Klausur mit 5.0 bewertet.

1. Aufgabe: Versionierung / Versionsverwaltung (5 Punkte)

Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter *Optimistic Locking* / *Pessimistic Locking?*

Basisanwendung

Für die folgenden Aufgaben ist eine Anwendung zur Verwaltung von Teppichen als Basis gegeben, siehe die folgenden Seiten.

Klasse Teppichverwaltung

```
public class Teppichverwaltung {

  private Teppich[] teppiche = new Teppich[2];

  private void fuelleTeppiche() {
        this.teppiche[0] = new Teppich("111_high", true, 180, 300, 590);
        this.teppiche[1] = new Teppich("222_low", false, 150, 270, 490);
  }

  private void schreibeTeppicheInCsvDatei() {
        try {
              Creator creator = new CsvExportCreator();
              Export export = creator.factoryMethod();
              export.schreibeTeppicheInDatei(this.teppiche);
        }
        catch(Exception exc) {
              exc.printStackTrace();
        }
  }

  public static void main(String[] args) {
        Teppichverwaltung teppichverwaltung = new Teppichverwaltung();
        teppichverwaltung.fuelleTeppiche();
        teppichverwaltung.schreibeTeppicheInCsvDatei();
  }
}
```

Klasse Teppich

```
public class Teppich {

  private String artikelnummer;
  private boolean hochflorTeppich;
  private int breite;
  private int laenge;
  private int preis;

  public String gibAttributeAus(String trenner) {
        String ergebnis = this.getArtikelnummer() + trenner
              + this.isHochflorTeppich() + trenner
              + this.getBreite() + trenner
              + this.getLaenge() + trenner
              + this.getPreis();
        Return ergebnis;
  }

  // Konstruktor mit Parametern und get- und set-Methoden
  …
}
```

Klasse DatenCsvExport

```
import java.io.*;

public class DatenCsvExport extends Export{

  @Override
  public void schreibeTeppicheInDatei(Teppich[] teppiche)
        throws IOException{
        BufferedWriter aus
              = new BufferedWriter(new FileWriter("Teppiche.csv"));
        for(int i = 0; i < teppiche.length; i++) {
              aus.write(teppiche[i].gibAttributeAus(";") + "\n");
        }
        aus.close();
  }
}
```

Klasse Export

```
import java.io.IOException;

public abstract class Export {

  public abstract void schreibeTeppicheInDatei(Teppich[] teppiche)
         throws IOException;
    }
}
```

Klasse Creator

```
public abstract class Creator {

  public abstract Export factoryMethod();
}
```

Klasse CsvExportCreator

```
public class CsvExportCreator extends Creator {

  @Override
  public Export factoryMethod() {
        return new DatenCsvExport();
  }
}
```

2. Aufgabe: Design Patterns (40 Punkte)

Grundlage ist die Basisanwendung zur Verwaltung von Teppichen

a) Geben Sie das UML Klassendiagramm an, welches sich aus den Klassen der Basisanwendung zur Verwaltung von Teppichen ergibt. Sie brauchen packages, get- und set-Methoden, Konstruktoren und Exceptions nicht zu berücksichtigen.

b) Nennen und erläutern Sie dasjenige Design Pattern, welches in diesem Beispiel umgesetzt wird.
   i) Welche Klassen realisieren das Design Pattern, welche benutzen es?
   ii) Was ist der Nutzen des Design Pattern bezogen auf dieses Beispiel?
   iii) Nennen Sie ein Beispiel für eine Weiterentwicklung der Anwendung, welche durch dieses Design Pattern unterstützt wird. Geben Sie den Quellcode dieser Weiterentwicklung an (, Erweiterung des Design Pattern und auch die Benutzung der Erweiterung).

3. Aufgabe: Vertiefung Java (30 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Teppichen und eine weitere Klasse *OutdoorTeppich*. Packages und Importe werden hier nicht berücksichtigt.

```
public class OutdoorTeppich extends Teppich{

  private boolean wasserabweisend;

  @Override
  public String gibAttributeAus(String trenner) {
        String ergebnis = super.gibAttributeAus(trenner) + trenner
              + this.isWasserabweisend();
        return ergebnis;
  }

  // Konstruktor mit Parametern und get- und set-Methode
  …
}
```

a) Generics, for each

Geben Sie den Quellcode an, um in der Klasse *Teppichverwaltung* das Attribut *teppiche* vom Typ *Teppich[ ]* zu ersetzen durch ein Objekt vom Typ *ArrayList* unter Verwendung von Generics. Geben Sie weiterhin den neuen Quellcode der Methode *fuelleTeppiche* der Klasse *Teppichverwaltung* und der Methode *schreibeTeppiche-InDatei* der Klasse *DatenCsvExport* unter Verwendung einer for each Schleife an. (Mit *for each* ist nicht die Methode *forEach* gemeint!) In welcher Klasse ist weiterhin eine Änderung notwendig? (Nennen Sie nur den Klassennamen.) Importe brauchen Sie nicht zu berücksichtigen.

b) Generics mit Typeinschränkungen

Geben Sie den Quellcode einer Klasse *TeppichUtil* an, welche ein Generic *T* enthält, für welches nur ein Objekt vom Typ *Teppich* oder einer Ableitung von *Teppich* eingesetzt werden kann. Es gibt ein privates Attribut *teppichBasis* vom Typ *T*, einen Konstruktor, welcher das Attribut mit einem Wert belegt, und eine Methode *gibAttributeUndQuadratmeterTeppichBasisInKonsoleAus* ohne Parameter und Rückgabewert. Diese schreibt die Attribute von *T* und auch die Quadratmeter in die Konsole, siehe unten.

Ergänzen Sie die *main*-Methode der Klasse *Teppichverwaltung* folgendermaßen. Erstellen Sie ein Objekt *teppich* vom Typ *Teppich* und ein Objekt *outdoorTeppich* vom Typ OutdoorTeppich und zwei Objekte vom Typ *TeppichUtil* unter Verwendung der Objekte *teppich* und *outdoorTeppich* und erzeugen Sie die folgende Ausgabe.

```
333_high true 180 250 450
Anzahl Quadratmeter des Teppichs: 32400
444_low false 140 230 420 true
Anzahl Quadratmeter des Teppichs: 19600
```

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (25 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Teppichen.

JUnit 5

Ergänzen Sie den vorliegenden Quellcode eines JUnit Tests. Der Konstruktor der Klasse *Teppich* soll durch diese JUnit Testklasse daraufhin getestet werden, ob die Artikelnummer beim Kreieren eines Teppichs richtig gesetzt wird. Weiterhin soll der Konstruktor daraufhin getestet werden, ob bei der Eingabe eines negativen Preises eine *IllegalArgumentException* mit der Meldung *Der Preis darf nicht negativ sein.* geworfen wird. Benutzen Sie aus der Klasse *Assertions*
assertTrue(BooleanSupplier conditionSupplier, Supplier<String> messageSupplier),
assertEquals(String actual, String expected und
<T> assertThrows(Class<T> clazz, Executable executable).

```
import static org.junit.jupiter.api.Assertions.*;
import  org.junit.jupiter.api.*;
public class TeppichTest{

   // Ergaenzen!!!


   @AfterEach
   public void tearDown() throws Exception{
         // Ergaenzen!!!

   }

   @Test
   public void test() throws Exception{
         // Ergaenzen!!!









   }

   @Test
   public void testException() throws Exception{
         // Ergaenzen!!!





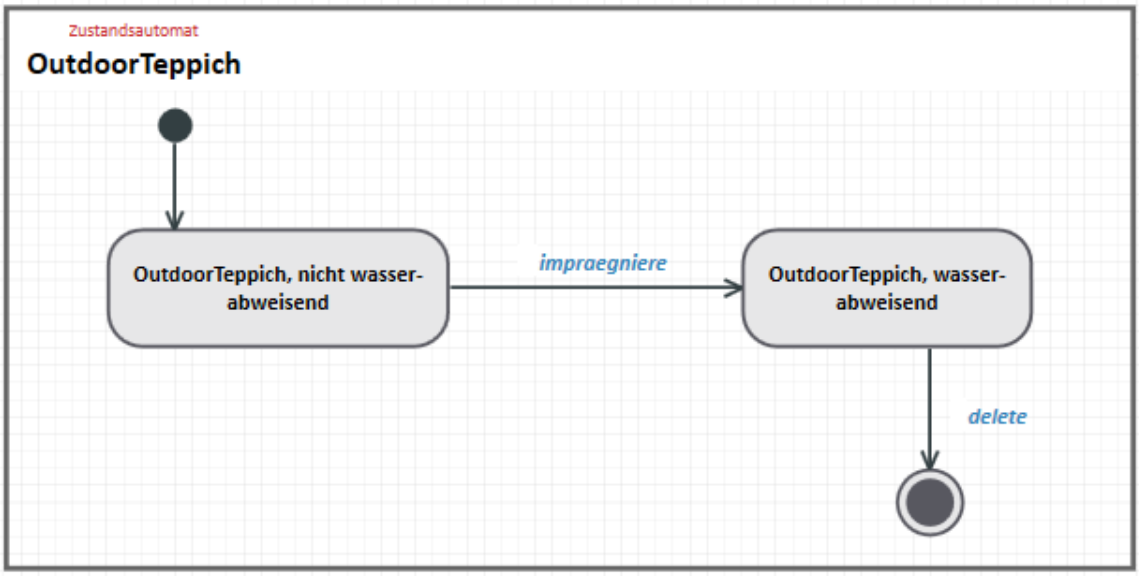




   }
}
```

Zustandsbasierter Test

Gegeben ist ein UML Zustandsautomat, welcher die Zustände eines Objekts vom Typ *OutdoorTeppich* der Aufgabe 3 modelliert. Geben Sie den Übergangsbaum an.
Geben Sie zwei Testfälle inklusive Sollergebnis an, die sich aus dem Übergangsbaum ergeben.



5. Aufgabe: SOLID und Clean Code (20 Punkte)

Gegeben ist die Anwendung zur Verwaltung von Teppichen inklusive der Klasse *OutdoorTeppich* aus der 3. Aufgabe.

a) Erläutern Sie das *Liskov Substitution Principle.* Nennen Sie ein Beispiel unter Verwendung der Klassen *Teppich* und *OutdoorTeppich*.

b) Wieso trägt das *Liskov Substitution Principle* zu einer höheren Wartbarkeit und Lebensdauer von Software und zu einer Reduktion der Fehleranfälligkeit bei (2-3 Sätze)?

c) Nennen und erläutern Sie kurz einen Wert aus Clean Code. Geben Sie an, ob, und wenn ja, wie in dem vorgegebenen Beispiel zur Verwaltung von Teppichen in den Aufgaben 2, 3 oder 4 eine Maßnahme zur Erfüllung dieses Werts umgesetzt wurde.

d) Erläutern Sie kurz das Prinzip *Integration Operation Segregation Principle (IOSP)*, welches dem roten Grad aus Clean Code zugeordnet ist. Was ist mit *Integration* gemeint, was mit *Operation*? Ist das Prinzip bereits in dem Beispiel zu *Teppichen* umgesetzt worden? Begründen Sie kurz Ihre Antwort.